



← Back to student guides

✦ Airflow

Orchestration

3 levels

113 sections



The Complete Guide

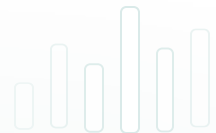
Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

DAG

SCHEDULER

EXECUTOR

WORKER



Part one of three. Almost every beginner problem with Airflow comes from one of three things: misunderstanding the interval model, putting expensive work at the top level of a DAG file, or forgetting that a task must be safe to run twice. Start with the error table, then work through the habits and practice cards underneath it.





Symptom	Real cause	Fix
The DAG does not appear in the UI	Import error, or no DAG object at module level	<code>python dags/my_dag.py</code> , then <code>airflow dags list-import-errors</code>
The DAG appears but never runs	It is paused, or <code>start_date</code> is in the future	Unpause it; check <code>start_date</code>
Hundreds of runs appeared at once	<code>catchup=True</code> with an old <code>start_date</code>	Set <code>catchup=False</code> ; backfill deliberately
The <code>@daily</code> run for today has not happened	The interval must end before the run starts	Correct behaviour: wait until after midnight
A rerun produced different data	The task used <code>datetime.now()</code>	Use <code>{{ ds }}</code> and the interval variables
A retry duplicated rows	The task is not idempotent	<code>DELETE</code> the target partition, then <code>INSERT</code>
<code>{{ ds }}</code> appears literally in the output	The field is not templated	Check the operator's <code>template_fields</code>
Airflow feels slow, and new DAGs appear late	Expensive top-level code	Move it inside a task; check Admin → DAG Processing
<code>Variable.get()</code> hammering the database	Called at module level	Move it into a task, or use <code>{{ var.value.x }}</code>
The whole cluster deadlocks while waiting	Sensors in <code>poke</code> mode holding worker slots	<code>mode="reschedule"</code> and a <code>timeout</code>
A sensor waits forever	No <code>timeout</code> set	Always set one
A task after a branch was skipped silently	Skips propagate; the join needs a trigger rule	<code>TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS</code>



Cleanup did not run after a failure	Default rule requires all upstreams to succeed	<code>TriggerRule.ALL_DONE</code>
The UI is slow and XCom pages hang	Large objects pushed to XCom	Write to storage; pass the path
<code>KeyError</code> pulling an XCom	Wrong <code>task_ids</code> or <code>key</code>	Check the XCom tab on the producing task
Tasks sit in <code>queued</code> and never start	Cluster or DAG concurrency limit reached	Check <code>parallelism</code> and <code>max_active_tasks</code>
A hung task blocked everything for hours	No <code>execution_timeout</code>	Set one on every task
Intervals piled up behind a stuck run	No <code>dagrun_timeout</code> , no <code>max_active_runs</code>	Set both
Two runs corrupted the same table	Overlapping runs	<code>max_active_runs=1</code>
A credential appeared in a task log	You printed a value you fetched yourself	Never log secrets; use <code>conn_id</code> , not the password
<code>ModuleNotFoundError</code> in a task but not locally	The worker's environment differs from yours	Install the dependency in the image the worker uses
Renaming the DAG lost all its history	<code>dag_id</code> is the identity	Renaming creates a new DAG; migrate deliberately
A change to the DAG file did nothing	The scheduler had not re-parsed yet	Wait 30 seconds; check for an import error

The habits that pay off most



Always `catchup=False`

Unless you specifically want history. Then backfill on purpose with a command you can rate-limit.

Use `{{ ds }}` , never `now()`

The whole value of Airflow's interval model is reproducible reruns, and `now()` throws it away.



Make every task idempotent

Delete the target partition, then write it. Retries and backfills become free rather than dangerous.



Set `execution_timeout` everywhere

One hung task with no timeout can hold a worker slot indefinitely and stall the whole DAG.



Keep the top level empty

Only DAG definition and task wiring. Everything else runs every thirty seconds, forever.



Credentials in Connections

Reference a `conn_id` . Never a password in a DAG file, and never a secret in a log.



`airflow tasks test` for iteration

Foreground, instant, logs in your terminal, no database state. The fastest loop available.



Tags, owner, and `doc_md`

Free to add, and they are what makes a DAG findable and supportable six months later.

Practice cards

Short, self-contained exercises. Each takes a few minutes and leaves you with a fact you will not forget.



then read a log line printing both `{{ ds }}` and `$(date -I)`. Seeing them differ is the moment scheduling clicks.

02 Flood your own cluster

Set a two-year-old `start_date` with `catchup=True` on a scratch DAG and unpause it. Count the runs, then pause it immediately. Doing this once, deliberately, means never doing it accidentally.

03 Break idempotence

Write a task that `INSERT` s rows. Clear it and rerun. Watch the row count double. Then rewrite it as delete-then-insert and repeat.

04 Deadlock a worker pool

Create more `poke` -mode sensors than you have worker slots, all waiting on files that do not exist. Watch real tasks starve. Then switch to `reschedule`.

05 Cause the silent skip

Build a branch with a join that has the default trigger rule. Watch the join and everything after it get skipped with no error anywhere.

06 Measure top-level cost

Note your parse duration in **Admin** → **DAG Processing**. Add `time.sleep(3)` at module level. Watch the number move. Move it inside a task and watch it recover.

07 Abuse XCom

Return `list(range(500_000))` from a task and observe the run duration and the XCom page. Then pass a file path instead and compare.

08 Kill a hung task

Add a task running `sleep 300` with `execution_timeout=timedelta(seconds=20)`. Watch it be killed and read the message.



and ``up_for_reschedule``. Recognising all six on sight is worth the ten minutes.

Debugging order

Follow this rather than guessing. The first three steps answer most problems.

01 Does the file import?

`python dags/my_dag.py` . Then `airflow dags list-import-errors` . A DAG that does not appear is almost always this.

02 Read the task log

Click the failed task, then **Logs**. Scroll to the `top` of the traceback rather than the bottom. The last line is usually a wrapper, not the cause.

03 Check the rendered template

The **Rendered Template** tab shows what your field became. Half of "my SQL is wrong" is a template that resolved differently than expected.

04 Run the task by hand

`airflow tasks test my_dag my_task 2024-05-01` . Foreground, instant, no state written. If it fails here too, you are debugging your code and not Airflow.

05 Check the state, not just the colour

`queued` means a capacity problem, `upstream_failed` means look upstream, `skipped` means a branch or trigger rule. Each points somewhere different.

06 Check the DAG's own health



BASH

Copy

```
python dags/my_dag.py # import check
airflow dags list-import-errors
airflow tasks test my_dag my_task 2024-05-01 # instant, no DB state
airflow dags test my_dag 2024-05-01 # a whole run, locally
airflow tasks list my_dag --tree # the structure Airflow sees
airflow dags report # parse duration per file
```

Read tracebacks from the top

Airflow wraps task execution in several layers, so the last lines of a failure are usually Airflow's own frames. The cause is higher up, in your code. Scrolling up is a habit that saves a lot of time.

The interval model, in practice

This is where most beginner time is lost, so start with these three questions.

"Why has today's run not happened?" Because the interval has not ended. A `@daily` DAG produces the run for 1 May just after midnight on 2 May.

"Which date should my task use?" `{{ ds }}`, the logical date, which is the start of the interval. Never the wall clock.

"How do I run yesterday again?" Clear that task instance. It will rerun with the original `{{ ds }}`, which is why idempotence matters.

You want	Use
The interval's date	<code>{{ ds }}</code> → 2024-05-01



The same, no dashes	<code>{{ ds_nodash }} → 20240501</code>
The interval bounds	<code>{{ data_interval_start }} / {{ data_interval_end }}</code>
Seven days back	<code>{{ macros.ds_add(ds, -7) }}</code>
A different format	<code>{{ macros.ds_format(ds, "%Y-%m-%d", "%d/%m/%Y") }}</code>
The run's identity	<code>{{ dag_run.run_id }}</code>

`datetime.now()` in a task is a reproducibility bug

It will produce correct-looking results today and wrong ones on every rerun and every backfill. The failure is silent: no error, just data for the wrong window written into the right partition. Grep your DAGs for `now()` and `today()` ; each occurrence is either a deliberate exception or a bug.

Idempotence, concretely

"Safe to run twice" is easy to say and easy to get wrong. Three patterns cover almost everything.

delete-then-insert (the default choice)

Copy

```
DELETE FROM daily_stats WHERE day = '{{ ds }}';
INSERT INTO daily_stats SELECT '{{ ds }}', count(*) FROM events
WHERE event_date = '{{ ds }}';
```

merge / upsert: when you cannot delete

Copy

```
MERGE INTO daily_stats t
USING (SELECT '{{ ds }}' AS day, count(*) AS n FROM events
WHERE event_date = '{{ ds }}') s
```



overwrite a partition (for files)

Copy

```
@task
def write_partition():
    path = "s3://analytics/events/dt={{ ds }}/"
    # write mode "overwrite" on the partition, not "append"
    df.write.mode("overwrite").parquet(path)
```

✓ Safe to retry

- `DELETE` the partition, then `INSERT`
- `MERGE` / upsert on a key
- Overwrite a partition path
- `CREATE OR REPLACE`
- Writing to a path derived from `{{ ds }}`

✗ Dangerous to retry

- Bare `INSERT` : duplicates on retry
- `UPDATE ... SET n = n + 1` : accumulates
- Appending to a file
- Sending an email or a webhook
- Writing to a path derived from `now()`

For non-idempotent side effects (sending a notification, charging a card) the pattern is to make the `decision` idempotent: record in a table that the notification for `{{ ds }}` was sent, and check that before sending.

Structuring a DAG that ages well

dags/pipeline.py

Copy

```
from datetime import datetime, timedelta
from airflow.decorators import dag, task

DEFAULTS = {
    "owner": "data-platform",
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
```



```
@dag(
    dag_id="pipeline",
    start_date=datetime(2024, 1, 1),
    schedule="0 2 * * *",
    catchup=False,
    default_args=DEFAULTS,
    dagrun_timeout=timedelta(hours=2),
    max_active_runs=1,
    tags=["production", "events"],
    doc_md="""
    ### Events pipeline
    **Owner:** data-platform · **Consumers:** exec dashboard
    **On failure:** idempotent – safe to clear and rerun.
    **Escalate if:** not green by 07:00 UTC.
    """,
)
def pipeline():
    ...
    pipeline()
```

Rule	Why
One responsibility per task	A change reruns the minimum, and failures are precise
Business logic in a plain function, called by the task	It becomes unit-testable without Airflow
SQL in <code>.sql</code> files, not Python strings	Reviewable, syntax-highlighted, and templated automatically
Task ids as verbs: <code>extract</code> , <code>validate</code> , <code>load</code>	The Graph view reads as a sentence
<code>default_args</code> for the common case, overrides per task	Readable, and consistent by default
<code>doc_md</code> with owner, consumers, and escalation	The runbook is where the on-call person already is

Keep the DAG file thin



cost you parse time.

Local development

BASH

Copy

```
# Get an Airflow you can break
curl -Lf0 'https://airflow.apache.org/docs/apache-airflow/stable/docker-compose.yaml'
mkdir -p dags logs plugins config
echo -e "AIRFLOW_UID=$(id -u)" > .env
docker compose up airflow-init && docker compose up -d

# The fast loop – no UI needed
airflow tasks test my_dag my_task 2024-05-01
airflow dags test my_dag 2024-05-01

# Before you commit
python dags/my_dag.py
airflow dags list-import-errors
```

Habit	Payoff
<code>airflow tasks test</code> over the UI	Seconds instead of a minute, and logs in your terminal
<code>python dags/my_dag.py</code> before committing	Catches the most common breakage instantly
A scratch DAG with <code>schedule=None</code>	Trigger it on demand while experimenting
<code>tags=["dev"]</code> on experiments	Filter them out of the production view
Small <code>retries</code> while developing	You do not want three retries of a broken task

Your local environment is not the worker's environment



fix is adding the dependency to the image the workers run, not to your laptop.

Small things worth doing from day one

Set `tags` on every DAG. The UI filter is the only thing that makes fifty DAGs navigable.

Put the owner in `default_args`. It shows in the UI and answers "who do I ask?" without a search.

Write `doc_md`. Even three lines. It renders in the UI where the on-call person is already looking.

Prefer `EmptyOperator` bookends. A `start` and `end` task make the Graph view readable and give you a stable place to attach new work.

Never rename a `dag_id` casually. It is the identity: renaming abandons all history and creates a new DAG with none.

Grep for `now()` before every review. Each occurrence is a reproducibility bug or a deliberate exception, and there is no third option.

Check `dvc -style: does a second run change anything?` Run your pipeline twice on the same interval and diff the output. If it differs, the task is not idempotent.

PYTHON

Copy

```
# A starter default_args worth copying
DEFAULTS = {
    "owner": "your-team",
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
    "max_retry_delay": timedelta(minutes=30),
```



A starter DAG worth keeping

Every line here is something from this page.

dags/starter.py

Copy

```
"""
### Starter pipeline
**Owner:** your-team · **Consumers:** nobody yet
**On failure:** idempotent – safe to clear and rerun any interval.
"""
from datetime import datetime, timedelta

from airflow.decorators import dag, task
from airflow.operators.empty import EmptyOperator
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
from airflow.sensors.filesystem import FileSensor
from airflow.utils.trigger_rule import TriggerRule

DEFAULTS = {
    "owner": "your-team",
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
    "execution_timeout": timedelta(minutes=30), # no task hangs forever
}

@dag(
    dag_id="starter",
    start_date=datetime(2024, 1, 1),
    schedule="0 2 * * *",
    catchup=False, # backfill deliberately, never by accident
    default_args=DEFAULTS,
    dagrun_timeout=timedelta(hours=2), # the whole run has a ceiling
    max_active_runs=1, # one writer per table
    tags=["starter", "dev"],
    doc_md=__doc__, # the runbook, in the UI
)
```



```
start = EmptyOperator(task_id="start")

wait = FileSensor(
    task_id="wait_for_input",
    filepath="/data/landing/events_{{ ds_nodash }}.csv",
    poke_interval=60,
    timeout=60 * 60 * 2,          # never wait forever
    mode="reschedule",          # release the worker slot while waiting
)

@task
def validate() → int:
    """Logic lives in a module; the task just calls it."""
    from mypkg.checks import count_rows          # imported inside the task
    count = count_rows(day="{{ ds }}")
    if count == 0:
        raise ValueError("no rows for {{ ds }}")
    return count                                # a number through XCom

# Idempotent: DELETE the partition, then INSERT. SQL in a file.
transform = SQLExecuteQueryOperator(
    task_id="transform",
    conn_id="warehouse",          # a Connection, never a password
    sql="sql/daily_rollup.sql",
)

cleanup = EmptyOperator(
    task_id="cleanup",
    trigger_rule=TriggerRule.ALL_DONE,  # runs whatever happened upstream
)

end = EmptyOperator(task_id="end")

start >> wait >> validate() >> transform >> cleanup >> end

starter()
```

sql/daily_rollup.sql

Copy

```
-- Safe to retry, safe to backfill.
DELETE FROM analytics.daily_events WHERE day = '{{ ds }}';
```



Nine details in there are the whole lesson of this page: `catchup=False` , `{{ ds }}` rather than `now()` , idempotent delete-then-insert SQL in a file, `retries` with backoff, `execution_timeout` per task and `dagrun_timeout` per run, `max_active_runs=1` , a sensor in `reschedule` mode with a timeout, `ALL_DONE` on cleanup, and a heavy import moved inside the task.

Mid-level tips go deeper on every one of these: executors and the concurrency limit that binds, dynamic mapping pitfalls, deferrable operators replacing sensors, pools for shared systems, alerting people read, testing DAGs in CI, and diagnosing "it works locally but not on the cluster".



PREVIOUS

Beginner · Interview Review

NEXT UP

Mid-level · Detailed



Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.





Courses

The MLOps Practitioner

Sessions

Articles

COMMUNITY

Meet the team

Services for companies

FAQ

Contact us

Privacy Policy

WHERE WE ARE

WhatsApp

Discord

LinkedIn

X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**